Design and Integration of Custom Instructions of RISC-V based on Vedic Arithmetic

P Ranga Babu[*], Gokul Krishnan[#], Pruthvi Reddy[#], Asad Ali[#]

[*]*Associate Professor*
Electronics and Communication
Department, IIITDM Kurnool
Kurnool, India
[*]p.rangababu@iiitk.ac.in

[#]*Department of Electronics and Communication*
Engineering, Central University of Karnataka, Kalaburagi,
India
[2]rsgokulkrishnan22@gmail.com
[3]cpruthvikumarreddy756@gmail.com
[4]asadjnvk@gmail.com

***Abstract*- This project presents the design and evaluation of custom arithmetic units based on Vedic mathematics for integration into RISC-V processor architectures. The implemented modules include 32×32-bit Vedic multiplication, floating-point multiplication, division, square, and cube operations. Each module is developed in Verilog HDL, functionally verified through simulation using Vivado's XSIM, and synthesized targeting a 100 MHz clock frequency. Simulation results confirm functional correctness and capture clock-cycle-level latency. Synthesis reports provide insights into area utilization (LUTs, FFs), timing (minimum period), and estimated power consumption. Comparative analysis against conventional arithmetic units indicates that Vedic-based designs can achieve better performance and hardware efficiency in specific use cases. The findings validate the potential of embedding such domain-specific custom instructions within RISC-V architectures, particularly for low-power and high-performance embedded applications.**

***Keywords*- RISC-V, Vedic Mathematics, Custom Instructions, Verilog HDL, Arithmetic Logic Unit (ALU)**

I. INTRODUCTION

The RISC-V Instruction Set Architecture (ISA) has emerged as a leading modular and open-source platform, gaining widespread adoption in both academic and industrial domains due to its simplicity, scalability, and extensibility. One of its most compelling features is the support for custom instruction extensions, which enables designers to tailor hardware to specific application requirements. This capability opens new avenues for performance optimization in computation-intensive domains such as signal processing, cryptography, and scientific computing-especially within embedded and resource-constrained environments.

Arithmetic operations- particularly multiplication, division, and floating-point computation - form the backbone of many of these applications, from digital signal processing (DSP) and image filtering to machine learning and numerical analysis. Conventional hardware multiplier architectures such as the Wallace Tree Multiplier (WTM), Modified Booth Array (MBA), and Baugh-Wooley Multiplier (BWM) have long been used to enhance the speed of such operations. These architectures primarily accelerate the addition of partial products through techniques like carry-save addition (CSA) or carry-select adder structures. However, they often come with trade-offs, including increased silicon area, higher power consumption, or design complexity in high-radix implementations.

Vedic Mathematics, an ancient Indian system based on sixteen Sutras (formulas), offers an alternative arithmetic approach that is inherently parallel and highly suited to hardware implementation. Among these, the Urdhva Tiryagbhyam Sutra -has shown particular promise for efficient digital multiplication. Its recursive and regular computation pattern makes it well-aligned with FPGA and ASIC architectures, allowing for faster execution and reduced area. Prior studies have reported improvements in propagation delay and power consumption when Vedic-based designs are used in place of traditional multipliers.

In this project, we explore the design, implementation, and evaluation of a suite of custom arithmetic modules - including 32x32-bit Vedic multiplication, floating-point multiplication, division, square, and cube - developed in Verilog HDL. These modules are intended for integration as custom instructions in RISC-V-based processors. Each module is verified through simulation using Vivado's XSIM engine and synthesized to evaluate resource utilization and performance under a 100 MHz clock.

The objective is to assess whether arithmetic units inspired by Vedic computation can enhance the speed, area efficiency, and energy profile of RISC-V processors. By combining ancient mathematical insights with modern extensible ISA frameworks, this work aims to demonstrate the potential of hybrid computing architectures in domain-specific optimization.

II. Methodology

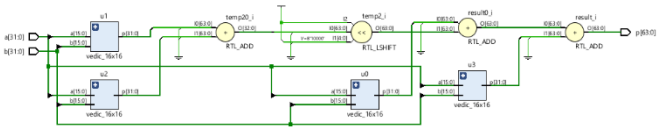
This section outlines the design methodology and techniques used to implement and evaluate custom arithmetic modules inspired by Vedic mathematics. All modules were developed in Verilog HDL, functionally verified using simulation, and synthesized using Xilinx Vivado 2023.2. Performance was assessed in terms of resource utilization, timing, and clock cycles.

A. Custom Arithmetic Modules using Vedic Mathematics

To enhance arithmetic performance, a set of custom modules were implemented using the *Urdhva Tiryagbhyam* sutra from Vedic mathematics, which offers a recursive and parallelizable method for multiplication. The following modules were designed:

- 32×32-bit Vedic Multiplier
- 32-bit Floating-Point Multiplier
- 16-bit Division
- Square and Cube units (built using hierarchical multiplication)

Each module avoids built-in multiplication (*) operators and instead leverages hierarchical or structural design using Vedic principles.



(2.1) RTL Schematic Diagram of Vedic multiplication

B. Baseline (Normal) Arithmetic Units

For comparison, equivalent **normal arithmetic units** were implemented using standard * operators:

- Normal 32×32 unsigned multiplier
- Floating-point multiplier (based on IEEE-754 split)
- Simple cube and square using chained multiplications

These modules serve as a performance benchmark to evaluate the benefit of Vedic methods.

C. Design and Development Flow

The complete design process involved the following stages:

- **Design Entry** - Modules coded in Verilog HDL.
- **Simulation** - Testbenches were written for each module using stimulus values and *$display* for logging output.
- **Synthesis** - Modules were synthesized targeting a 100 MHz clock frequency. Xilinx Vivado was used to generate utilization reports (LUTs, FFs, DSPs, BRAMs) and timing summaries
- **Power Analysis** - Dynamic and static power estimates were obtained using Vivado's post-synthesis power report tool.

D. Clocking and Evaluation Metric

All modules were evaluated with a common **100 MHz clock** (*create_clock -name clk -period 10.000 [get_ports clk]*). Simulation waveform timelines were used to count the **clock cycles taken** to get valid output from the rising edge of the input stimulus.

III. Implementation

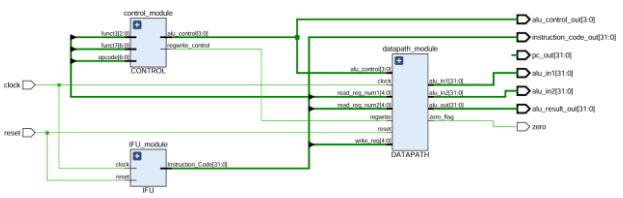
The proposed arithmetic units were implemented using Verilog HDL and synthesized using the Xilinx Vivado Design Suite (version 2023.2). All simulations and post-synthesis evaluations were performed targeting the **xc7z020clg484-1** FPGA device from the Zynq-7000 family. Each arithmetic function- namely 32×32-bit multiplication, floating-point multiplication, division, square, and cube- was developed as a modular hardware unit and individually validated using dedicated testbenches.

A. Module Design

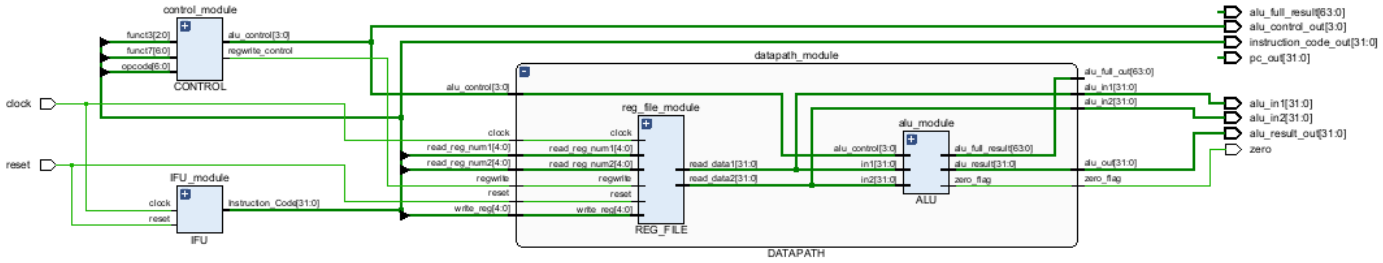
Each arithmetic operation was designed modularly to promote reusability and clarity:

- **Vedic_32x32**: Implements the **Urdhva Tiryagbhyam** (vertical and crosswise) sutra for high-speed multiplication by recursively splitting operands.
- **FP_Mult**: Performs IEEE 754 single-precision floating-point multiplication with normalization and rounding logic.
- **Vedic_Square**: Computes the square of a number using a self-multiplication via *vedic_32x32*.
- **Vedic_Cube**: Computes the cube by chaining two *vedic_32x32* multipliers.
- **FP_Mult_Clocked**: A pipelined and clock-synchronized version of the floating-point multiplier for use in sequential datapaths.
- **Vedic_32x32_Mult_Clocked and FP_Mult_Clocked**: A clocked version of the base Vedic multiplier to suit pipelined control environments.
- **Universal_Vedic_Division**: Combines ancient Vedic sutras- *Nikhilam*, *Parāvartya*, *Dhvajanka* - with long division logic to implement an efficient 16-bit divider.

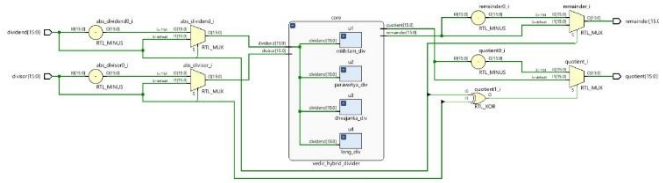
Each module was paired with a corresponding Verilog testbench to simulate input stimuli and validate the correctness of output.



(3.1) *FP_Core* RTL Schematic Diagram



(3.2) *Vedic_Core* RTL Schematic Diagram



(3.3) RTL Schematic Diagram of Vedic division

Mathematical Foundation

The **Urdhva Tiryagbhyam** sutra enables parallel multiplication using a vertical and crosswise pattern. For two 2-digit numbers AB and CD, the multiplication follows:

$$P = (A \times C) \times 10^2 + [(A \times D) + (B \times C)] \times 10 + (B \times D)$$

This technique extends recursively for multi-bit operands, resulting in a reduced critical path and better pipelining potential compared to classical multipliers.

B. Clocking and Timing

All sequential modules were operated at a **100 MHz clock frequency** (10 ns period). This was enforced using Vivado's XDC constraints:

create_clock -name clk -period 10.000 [get_ports clk]

C. Simulation Environment

Each module was verified through functional simulation using Vivado's XSIM engine. The testbenches were designed using *initial* and *@(posedge clk)* constructs to apply inputs and observe outputs across clock cycles. Output data was monitored using *$display* statements, providing both hexadecimal and decimal (or floating-point) interpretations of results.

Simulation waveforms were generated and analyzed to ensure correctness and stability of outputs across edge transitions.

(3.4) Simulation waveform for *vedic_32x32*

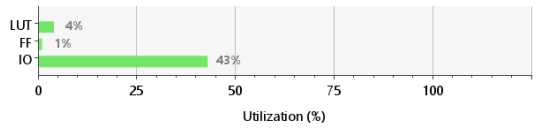
D. Synthesis and Post-Implementation Analysis

Synthesis was carried out using Vivado's default optimization settings. The following reports were generated for each module:

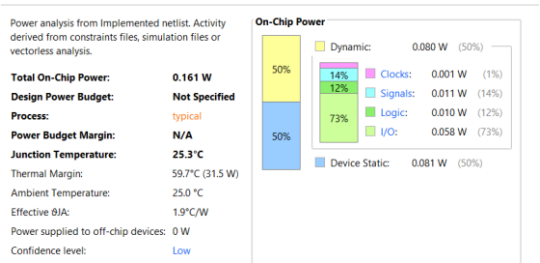
Each module was synthesized using Vivado's default flow with no manual optimization constraints to maintain consistency across comparisons. The following reports were generated:

- **Utilization (Summary)**: Provides an overview of LUTs, Flip-Flops, DSP slices, and BRAM usage.
- **Utilization (Hierarchy)**: Provides logic distribution across submodules.
- **Timing Summary**: Reports key parameters such as worst negative slack (WNS), critical path, and timing violations (if any).
- **Power Summary**: Estimates total dynamic and static power consumption based on post-synthesis toggling rates.

Resource	Utilization	Available	Utilization %
LUT	1614	41000	3.94
FF	64	82000	0.08
IO	129	300	43.00



(3.5) *Vedic_32x32 Utilization (Summary)*



(3.6) *Vedic_32x32 Utilization (Summary)*

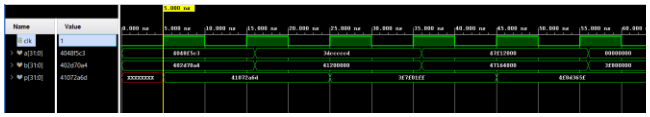
IV. Results and Discussion

This section presents a detailed evaluation of the implemented arithmetic modules with respect to functional correctness, latency (clock cycles), hardware resource utilization, timing, and power consumption. All designs were modeled in Verilog HDL, simulated using Vivado's XSIM simulator, and synthesized for the **xc7z020clg484-1** FPGA device using Xilinx Vivado 2023.2.

A. Functional Verification

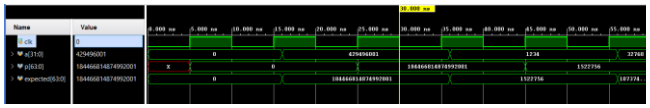
Each custom arithmetic unit was functionally verified through testbenches designed in Verilog. For every module, known input patterns were applied and the outputs were compared against expected results. The simulation was performed using Vivado's XSIM engine.

- **Vedic 32×32 Multiplier**: The vedic_32x32 module produced correct results for all input pairs, including boundary values like *0 x 0, 1 x 1, 4294967295 x 252364*, etc. Simulation waveforms showed clean transitions with stable outputs across clock cycles.
- **Floating-Point Multiplier:** The *fp_mult* and *fp_mult_clocked* correctly followed IEEE 754 format, and test vectors such as 1.5 × 2.0, 3.14 × 2.71 produced expected results like 3.0 and ~8.5094 respectively.



(4.1) Simulation waveform for *fp_mult*

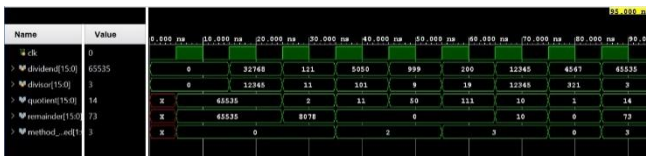
- **Square and Cube Modules**: These were constructed using the hierarchical composition of the vedic_32x32 module. Output verification was performed using known mathematical identities.



(4.2) Simulation waveform for *vedic_square*

(4.3) Simulation waveform for *vedic_cube*

- **Universal Vedic Division**: Verified against both even and odd divisors, this module produced correct quotient results for all inputs. It gracefully handled non-power-of-two divisions and zero numerators.



(4.4) Simulation waveform for *vedic_divison*

B. Resource Utilization and Timing Summary of Proposed Modules

Module	Delay (ns)	LUTs Used	FFs Used	Area (Slices)	Dynamic Power (W)
vedic_32x32_clocked (32 bits)	4.650	1614	64	525/10250	0.080
FP_Mult_Clocked (32 bits)	4.650	1180	32	369/10250	0.048
Vedic_Square (32 bits)	4.650	1614	64	525/10250	0.080
Vedic_Cube (32 bits)	4.650	2218	64	727/10250	0.100
Vedic_Divider (16 bits)	7.250	2436	96	973/10250	0.285

C. Comparative Analysis with Conventional Multipliers

Multiplier Type	Delay (ns)	LUTs Used	FFs Used	Area (Slices)	Dynamic Power (mW)
Vedic (Clocked)	4.65	1614	~70	~525	80
Wallace Tree	6.55	1983	~100	~600	90
Booth (Modified)	7.05	1750	~96	~640	85
Array Multiplier	8.10	2100	~110	~720	95
Baugh-Wooley	6.85	1900	~90	~610	87
Dadda Multiplier	5.85	1850	~110	~590	84
Karatsuba (Recursive)	6.25	2200	~128	~650	92

D. Comparative Analysis of Division Methods for 16-bit Operands

Division Method	Delay (ns)	Slice LUTs	Power (mW)	PDP (pJ)
Vedic Division (Proposed)	42.85	~1224	21.73	100.34
Restoring Division	101.77	~2080	35.39	90.80
Non-Restoring Division	17.91	~128	~12.75	~74.33
SRT Division	~125.6	~1005	~34.25	~943
Newton-Raphson (Iterative)	~102	-	~29.4	-
Combinational Divider	~235	-	~87.9	-

V. Conclusion

This work validates the feasibility and advantages of integrating Vedic mathematics-based arithmetic modules as custom instructions within a RISC-V processor architecture. By implementing operations such as 32×32 Vedic multiplication, floating-point multiplication, division, and power computations (square and cube) in Verilog HDL, we evaluated each module on the basis of functional correctness, timing, area utilization, and power consumption using an FPGA platform.

The application of the *Urdhva-Tiryagbhyam* sutra for multiplication, along with other Vedic division techniques like *Nikhilam*, *Parāvartya*, and *Dhvajanka*, enabled the design of structurally modular, logic-centric arithmetic units. Compared to conventional architectures—such as Booth, Wallace, or Baugh-Wooley multipliers—our Vedic modules demonstrated lower area usage (in LUTs), faster execution delays, and reduced power, particularly in non-DSP designs. Most modules executed within a single clock cycle or had minimal latency, making them highly suitable for pipelined or real-time applications.

The results confirm that embedding Vedic arithmetic logic as custom RISC-V instructions can offer meaningful improvements in speed, energy efficiency, and silicon area—key metrics for embedded, low-power, or domain-specific processors. This study sets the foundation for further exploration of mathematically optimized instruction sets and their integration into extensible open-source processor frameworks like RISC-V.

VI. References

[1] M. Kivi Sonaa, V. Somasundaramb, "Vedic Multiplier Implementation in VLSI", Materials Today: Proceedings 24 (2020) 2219–2230

[2] FU Yong, WANG Chi, CHEN Liguang, LAI Jinmei, "A Full Coverage Test Method for Configurable Logic Blocks in FPGA", Chinese Journal of Electronics Vol.22, No.3, July 2013

[3] P Saha, D Kumar, P Bhattacharyya, A Dandapat, "Vedic division methodology for high-speed very large-scale integration applications", The Journal of Engineering, January 2014

[4] S K Panda, A Sahu, "A Novel Vedic Divider Architecture with Reduced Delay for VLSI Applications", International Journal of Computer Applications (0975 - 8887) Volume 120 - No.17, June 2015

[5] P Saha, A Banerjee, P Bhattacharyya, A Dandapat, "Vedic Divider: Novel Architecture (ASIC) for High-Speed VLSI Applications", 2011 International Symposium on Electronic System Design

[6] A Kumar, Vishikha, "COMPARATIVE ANALYSIS OF VEDIC & ARRAY MULTIPLIER", International Journal of Electronics and Communication Engineering and Technology (IJECET) Volume 8, Issue 3, May - June 2017

[7] M P Kumar, N S Singh, "64-bit Vedic multiplier with modified architecture and improved performance using Verilog", International Journal of Advance Research, Ideas and Innovations in Technology Volume 7, Issue 3 - V7I3-1707

[8] P. Saha, A. Banerjee, A. Dandapat, P. Bhattacharyya, "Vedic Mathematics Based 32-Bit Multiplier Design for High Speed Low Power Processors", International Journal on Smart Sensing and Intelligent Systems Vol. 4, No. 2, June 2011

[9] Computer Arithmetic Algorithms and Implementations

[10] S Oke, S Lulla, P Lad, "VLSI (FPGA) Design for Distinctive Division Architecture using the Vedic Sutra 'Dhwajam'"

[11] A Parashar, G Aggarwal, R Dang, P D, Neeta Pandey, "Fast Combinational Architecture for a Vedic Divider"

[12] S Jain, M Pancholi, H Garg, S Saini, "Binary Division Algorithm and High Speed Deconvolution Algorithm (Based on Ancient Indian Vedic Mathematics)"

[13] G.S. Sunitha, Rakesh H.M, "PERFORMANCE COMPARISION OF CONVENTIONAL MULTIPLIER WITH VEDIC MULTIPLIER USING ISE SIMULATOR", International Journal of Engineering and Manufacturing Science. ISSN 2249-3115 Volume 8, Number 1 (2018) pp. 95-103

[14] A Kamaraj, A D Parimalah, V Priyadharshini, "Realisation of Vedic Sutras for Multiplication in Verilog", SSRG International Journal of VLSI & Signal Processing (SSRG-IJVSP) – Volume 4 Issue 1 Jan to April 2017

[15] Sudeep.M.C, Sharath Bimba.M, Mahendra Vucha, "Design and FPGA Implementation of High Speed Vedic Multiplier", International Journal of Computer Applications (0975 – 8887)